Point-in-Time Multi-Agent LLM Trading: A Reproducible LangGraph Stack with Ablations and Fair Backtesting

[Rohit Edward Martires] (Independent researcher, [Goa, India]). Email: [rohitmartires1

30 April 2026

Abstract

Large language models are increasingly used in financial decision pipelines, yet published agent stacks are often hard to reproduce, weakly specified at the tool-and-data boundary, and difficult to compare under controlled depth-of-reasoning ablations on historical "as of" dates. We present **Hindsight 20/20**, an open-source **LangGraph** implementation for single-asset trading research that composes specialist analysts, debate, and risk critique in one graph. Stages can emit **structured outputs** validated with **Pydantic**, improving auditability of intermediate artifacts. Market retrieval routes through a vendor layer enforcing a **simulation end date** for point-in-time access, with optional **ticker anonymization**. A **paper backtest** replays daily signals with configurable costs and metrics; **preset ablations** (`a1`–`full`) toggle pipeline depth via `PAPER_ABLATION` without forking code. Scripts and notebooks reproduce analyses from stored CSVs. We position the stack as a reproducible baseline under explicit temporal controls; **Langfuse**-compatible tracing supports operational debugging rather than a standalone empirical contribution.

Keywords: algorithmic trading; large language models; multi-agent systems; reproducibility; point-in-time data.

0.1 Introduction

Researchers and practitioners are experimenting with large language models (LLMs) as planners, analysts, and debaters inside trading and portfolio workflows. A single-asset, day-ahead setting is a natural testbed: the model must interpret noisy text and tabular context, call tools, and emit a discrete action whose quality can be checked later against realized prices. Yet building *research-grade* stacks in this space is still brittle. Systems are often described at a high level while leaving tool wiring, data routing, and temporal constraints implicit, which makes leakage across train/serving or "as of" boundaries hard to audit. Worse, when teams compare "more agents" vs "fewer agents," they frequently fork codepaths or change prompts ad hoc, so it is unclear whether measured differences come from added reasoning depth or from an unrelated implementation drift.

We target **historical, point-in-time** use: on each trade date the pipeline may only consume information available on or before that date. That requirement is easy to state and hard to enforce consistently once multiple vendor adapters, caches, and tool schemas are in play. We also care about **traceability**: downstream consumers should be able to inspect what each stage produced without scraping free-form blobs that are expensive to validate or diff across runs.

This paper presents **Hindsight 20/20**, an open-source implementation built on **LangGraph**, focused on *reproducibility and fair comparison* rather than on claiming a new state-of-the-art return. The same graph drives live UI/CLI runs and scripted backtests. Analyst and researcher modules gather context through a vendor layer governed by a **simulation end date**; **structured outputs** (Pydantic schemas with structured-output fallbacks) make stage artifacts inspectable. For studies that seek to reduce direct ticker exposure in model-

visible strings, optional **anonymization** can be enabled via configuration. **Preset ablations** (`a1`–`full`), selected through a single environment-controlled flag, vary which analysts and debate stages execute while importing the same engine code, so ablation studies do not require parallel forks. A **paper backtest** replays produced signals through a ledger with configurable fees and standard performance metrics, emitting CSV-style artifacts and pairing with notebooks for offline analysis.

Related literature is discussed in Section 2. Section 3 describes the system as implemented—graph orchestration, tools and data routing, structured outputs, temporal policy, anonymization, ablations, and the backtest harness. Section 4 reports a concrete evaluation protocol and results from stored CSV artifacts and companion notebooks (single ticker; expanded benchmarks deferred). Section 5 concludes with limitations honest to the current design. Section 6 lists contributions.

0.2 Related Work

LLM agents for trading and finance. A line of recent work studies autonomous or semi-autonomous LLM agents for market analysis and trading decisions. Xiao et al. introduce **TradingAgents**, a multi-agent framework in which specialized roles debate and coordinate toward trading outputs, illustrating how orchestrated LLM workflows can structure financial reasoning beyond a single chat completion (Xiao et al., 2024). Yu et al. propose **FinMem**, which combines layered memory and "character" design to steer an LLM trading agent, emphasizing long-horizon interaction patterns and task-specific scaffolding (Yu et al., 2023). Broader open financial LLM efforts such as **FinGPT** lower the barrier to experimenting with finance-tuned models and data pipelines in public repositories (Yang et al., 2023). These contributions are largely **algorithmic and modeling-focused**: they motivate multi-role graphs, tool use, and memory-rich agents. Our work is complementary: we foreground **reproducible systems engineering**—a single LangGraph codepath, **preset ablations** on that path, **schema-constrained stage outputs**, and a **scriptable paper backtest** so

that comparisons of pipeline depth are not confounded by forked implementations.

Point-in-time data, leakage, and memorization. Empirical finance and applied ML have long stressed that conclusions can be invalidated when features are accidentally built with future information or when identifiers leak test-set signal. Large models exacerbate the problem in two ways: tool-augmented agents can issue requests whose temporal semantics are easy to mis-specify across vendors, and strong LMs can exploit trivial cues (for example recurring ticker strings or corpus-regularities) unless experiments control what the model sees. Survey papers and venue-specific benchmarks are still catching up to fully standardized "as of" protocols for LLM agents; we therefore treat an explicit **simulation end date**, centralized vendor routing, and **optional ticker anonymization** as first-class implementation requirements rather than afterthoughts.

Multi-agent depth, cost, and fair ablation. Each additional analyst, debate round, or risk critique typically multiplies LLM calls and latency. Prior agent frameworks demonstrate that deeper graphs can improve flexibility but rarely ship a **small, named set of ablation modes** wired to one implementation, which makes it difficult to attribute benchmark shifts to architecture versus accidental prompt or tooling drift. Our stack adopts the multi-agent idiom in the spirit of TradingAgents- and FinMem-style systems, but emphasizes **controlled depth-of-pipeline studies**: toggling phases through configuration while sharing code, logs, and backtest drivers.

Execution realism and evaluation harnesses. Research prototypes often report model-side decisions without a transparent link to **fees, ledger state, and realized mark-to-market assumptions**—choices that dominate single-asset P&L in realistic settings. Open financial model stacks (Yang et al., 2023) improve accessibility of *models*; we instead document a **paper-ledger backtest** with configurable costs and standard summary metrics, so that agent outputs can be turned into equity curves and tables under stated assumptions. This does not simulate full limit-order-book microstructure; it **matches the fidelity of the**

shipped code and keeps evaluation claims auditable.

Positioning. Relative to Xiao et al. (2024), Yu et al. (2023), and Yang et al. (2023), Hindsight 20/20 does not introduce a new standalone monetary benchmark winner. It contributes an **open, memory-aligned implementation narrative**: multi-agent LangGraph orchestration, structured outputs, temporal and anonymization controls, preset ablations, and a fee-aware backtest harness suitable for reporting results tied to **concrete artifacts** (CSVs, logs, notebooks).

0.3 Method / System

This section summarizes the **shipped** implementation at the level of architecture and interfaces.

0.3.1 Architecture

The runtime centers on **`TradingAgentsGraph`** (`tradingagents/graph/trading_graph.py`), which wires LangGraph nodes defined by **`GraphSetup`** (`tradingagents/graph/setup.py`) and **`ConditionalLogic`** (`tradingagents/graph/conditional_logic.py`)—specialist analysts, optional bull/bear investment debate, trader synthesis, optional risk debate and judge, and signal extraction toward an executable instruction. Propagation per trade date uses **`Propagator`** (`tradingagents/graph/propagation.py`) with **`Reflector`** and **`SignalProcessor`** hooks as configured. The same graph class backs interactive CLI/UI flows and scripted **`run_backtest_mvp`** runs so backtests exercise the same orchestration path as exploratory sessions.

0.3.2 Agents, tools, and data routing

Analyst modules (market, social/news, fundamentals, etc.) consume tools implemented on **`tradingagents.agents.utils.agent_utils`**—price history, indicators, fundamentals line items, news, and related feeds—backed by vendor adapters and caches

under **tradingagents/dataflows/**. Configuration flows through **set_config** and **default_config.py** so provider choice and credentials remain centralized.

0.3.3 Structured stage outputs

Stages that call LLMs can request **schema-constrained** completions aligned with Pydantic models in **tradingagents/schemas/**, with **invoke_fallback** handling providers that do not reliably honor native structured output. Validated objects are merged into **LangGraph state**, which downstream nodes and logging can consume without ad hoc parsing of unstructured blobs—supporting design goals **S3** / **M3** even though we do not claim log-level empirical analyses here.

0.3.4 Point-in-time policy

effective_simulation_end_date_str and related helpers clamp vendor-facing reads so that, on date (d), tooling does not request post-(d) facts through the normal routing path (**M4**). Exact adapter semantics vary by source; the invariant we encode is an explicit simulation horizon wired through propagation rather than ad hoc date strings scattered across tools.

0.3.5 Optional ticker anonymization

When enabled, **TickerMapper** substitutes opaque identifiers for raw tickers in prompts and tool I/O suitable for anonymization-aware experiments (**M5**), reducing trivial symbol leakage without replacing fundamentals semantics inside adapters.

0.3.6 Preset ablations

PAPER_ABLATION selects among **a1**, **a2**, **a3**, and **full** (tradingagents/paper_ablation.py). **apply_paper_ablation_to_config** sets **selected_analysts**, **run_investment_debate**, and **run_risk_phase** in place:

Preset	Analysts	Investment debate	Risk phase
`a1`	market only	off	off
`a2`	market, social, news, fundamentals	off	off
`a3`	four analysts	on	off
`full`	four analysts	on	on

All presets share one codebase path (**S2** / **M2**).

0.3.7 Paper backtest harness

`run_backtest_mvp` (`tradingagents/backtest/runner.py`) iterates trading dates, calls **`graph.propagate`** per day, maps graph output to **`BUY`** / **`SELL`** / **`HOLD`** via **`SignalProcessor`** when **`use_llm_signal`** is enabled for ambiguous heuristics, and applies fills through **`PaperLedger`** with **`cost_model`** (`flat_bps`, **`zerodha_delivery`**, **`zerodha_intraday`**) and optional **`slippage_bps`** (**M6**). Scripts **`scripts/backtest_mvp.py`** and **`scripts/backtest_mvp_ablatio**
build configs from environment overrides (LLM provider, fee knobs, ablation flag). Schedule mode (**`--dates-csv`**) atomically updates a wide CSV (signals, equity path, per-stage outlook snapshots where emitted); standalone runs write **`equity.csv`**, **`trades.csv`**, and **`summary.json`** under **`eval_results/<ticker>/backtest_mvp_<id>/`**.

0.3.8 Observability (non-empirical)

When Langfuse credentials are present, runs attach correlation metadata for debugging (**optional trace hooks**); we do not treat trace dashboards as experimental evidence in Section 4.

0.4 Experiments

0.4.1 Goals and scope

We illustrate the stack with **small-scale, fully reproducible** runs aligned with a frozen artifact specification (**Frozen E1**) tied to the repository release: **one Indian equity listing (`RELIANCE.NS`)**, cost assumptions recorded in outputs (**e.g. `zerodha_delivery`** with **`slippage_bps=0`** in the longitudinal CSV header), and **preset ablations** over a fixed calendar window. We report **ending paper equity** from stored CSVs; Sharpe, drawdown, and richer summaries can be recomputed in notebooks via **`tradingagents.backtest.metrics`**. We **do not** claim breadth across universes, regimes, or alternative LLMs—the goal is a documented baseline tied to files in **`results/`**.

0.4.2 Protocol

Longitudinal schedule CSV. **`scripts/backtest_mvp.py`** was run in **`--dates-csv`** mode to produce **`results/dates.csv`**: one row per calendar row in the schedule, **`processed`** flags, **`final_signal`**, ledger columns, risk-adjusted summary columns when populated, and per-analyst **outlook** fields mirroring structured state for qualitative inspection. **`scripts/backtest_analysis.ipynb`** loads this file for plots.

Fixed-window ablations. **`scripts/backtest_mvp_ablations.py`** loops **`a1 … full`** over weekdays from **`2024-05-01`** through **`2024-06-28`**, writing **`results/{preset}_2024-05-01_2024-06-30_F`** (filename uses the requested range endpoints). The bundled inline driver sets **`initial_cash=100,000`**, **`buy_fraction=1.0`**, and **`use_llm_signal=False`**, i.e. no LLM **`SignalProcessor`** fallback when the heuristic mapper is ambiguous. **`scripts/backtest_ablation_analy`** overlays equity curves and tabulates metrics.

LLM. Frozen E1 runs used **`qwen/qwen3.5-flash-02-23`** (OpenRouter model slug) per the active **`scripts/backtest_mvp.py`** / environment configuration when those CSVs were produced; pin **`QUICK_THINK_LLM`**, **`DEEP_THINK_LLM`**, and provider keys in an appendix if

quick and deep models differ.

0.4.3 Results

Ablations (May–June 2024, `RELIANCE.NS`). Table 1 lists **ending portfolio value** after fees from the frozen CSVs (**initial cash $100,000**). Rankings are **not** stable extrapolations—two months on one name illustrate dispersion across pipeline depth only.

Table 1. Ending equity by ablation preset (frozen artifacts under **`results/`**).

Preset	Ending equity (notional)
`a1`	$111,480.32
`a2`	$110,159.53
`a3`	$89,749.01
`full`	$103,026.10

Under these runs, **more pipeline depth is not monotone in return**: the **`a1`** (single-analyst, no debates) path finishes highest and **`a3`** (debates without risk) finishes lowest, suggesting interactions between signal variance, turnover, and fees—consistent with the need for controlled ablations rather than informal "add agents" comparisons.

Longitudinal `dates.csv`. The schedule CSV spans additional calendar rows; the **last processed row with populated ledger fields** in our artifact ends `2024-07-29` with equity **$106,029.65** and reported **`total_return` approximately 6.03%** over the logged schedule fragment—use **`backtest_analysis.ipynb`** for full curves, turnover, and cost decomposition.

0.4.4 Figures

Publication figures were generated from the frozen CSVs using **`scripts/generate_paper_figures.py`** (also bundled with the repository). They appear below (Figures 1–8).

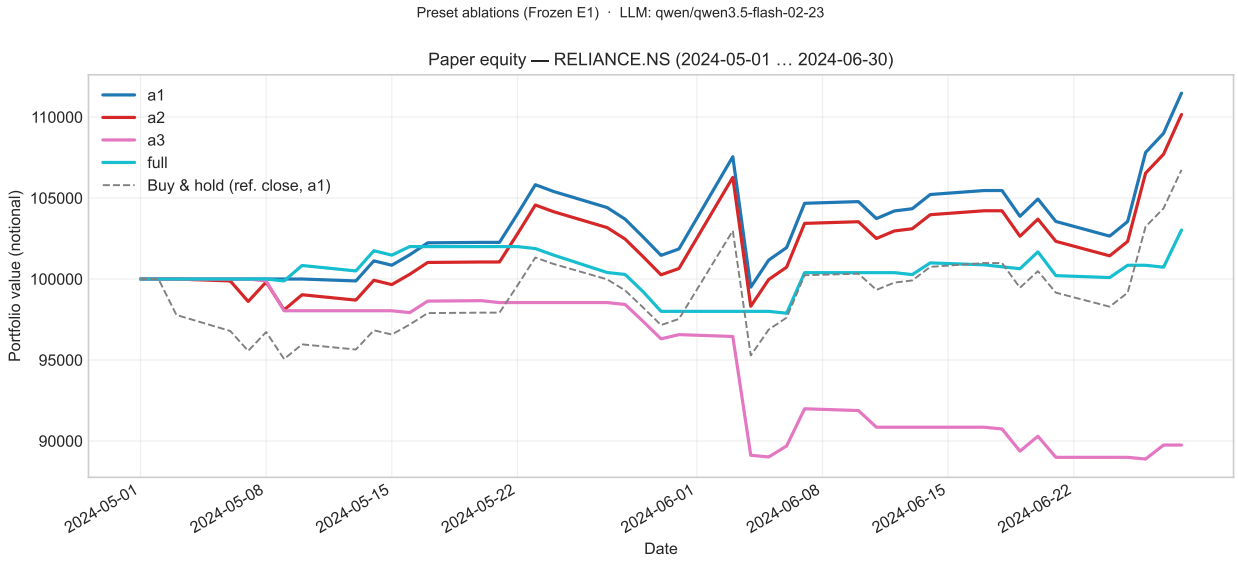

Figure 1: Figure 1. Equity curves by ablation preset with buy-and-hold reference (Frozen E1 window).

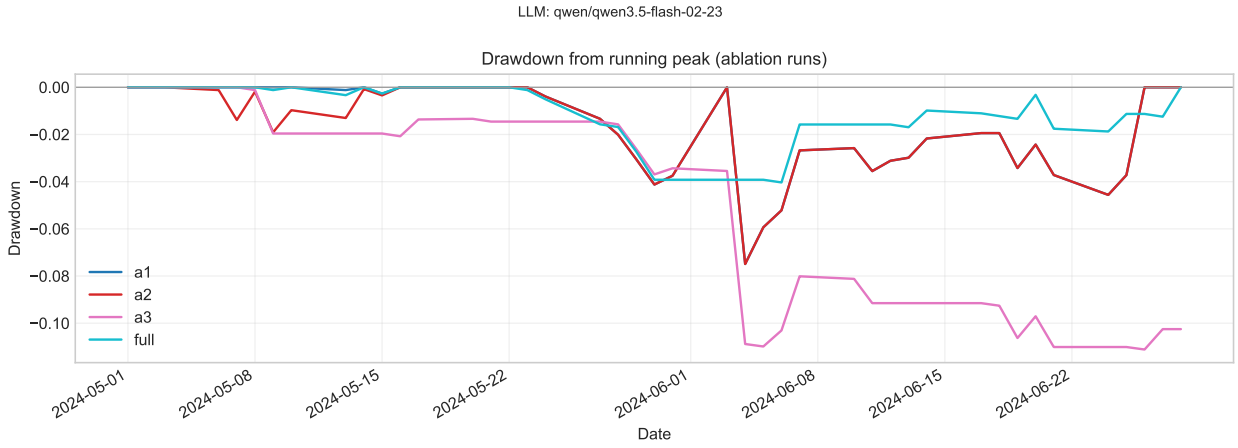

Figure 2: Figure 2. Drawdown from running peak by ablation preset.

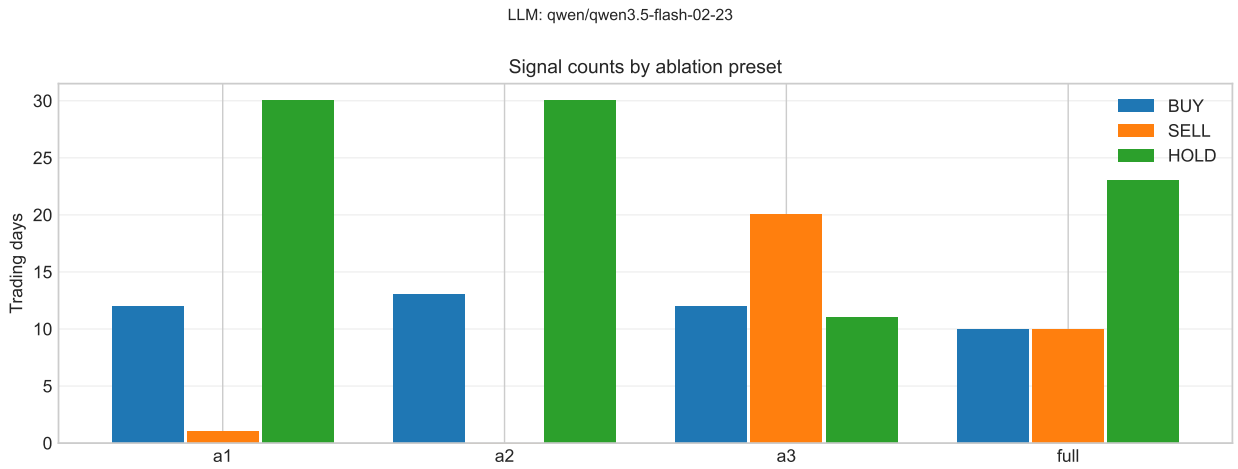

Figure 3: Figure 3. Signal counts (BUY / SELL / HOLD) by ablation preset.

0.5 Conclusion

We presented **Hindsight 20/20**, an open LangGraph implementation for single-stock LLM trading research with **preset ablations**, **structured outputs**, **simulation-date-aware data routing**, optional **ticker anonymization**, and a **fee-aware paper backtest** producing CSV artifacts and notebooks. Empirically, we documented **Frozen E1** runs on `RELIANCE.NS`, showing how ending equity varies across `a1`–`full` on a two-month window while a richer `dates.csv` trace supports qualitative inspection of stage outlooks.

Limitations. Evaluation is **narrow** (one ticker, short horizon, environment-specific LLM and vendors). `use_llm_signal` was **disabled** in the bundled ablation driver, so reported paths emphasize deterministic heuristic mapping unless separately enabled. We do **not** simulate microstructure, partial fills, or portfolio-wide constraints; `PaperLedger` is a research scaffold, not a broker simulator. Future work includes broader universes, model sweeps, and features tracked in the public roadmap once implemented—distinct from claims in this manuscript.

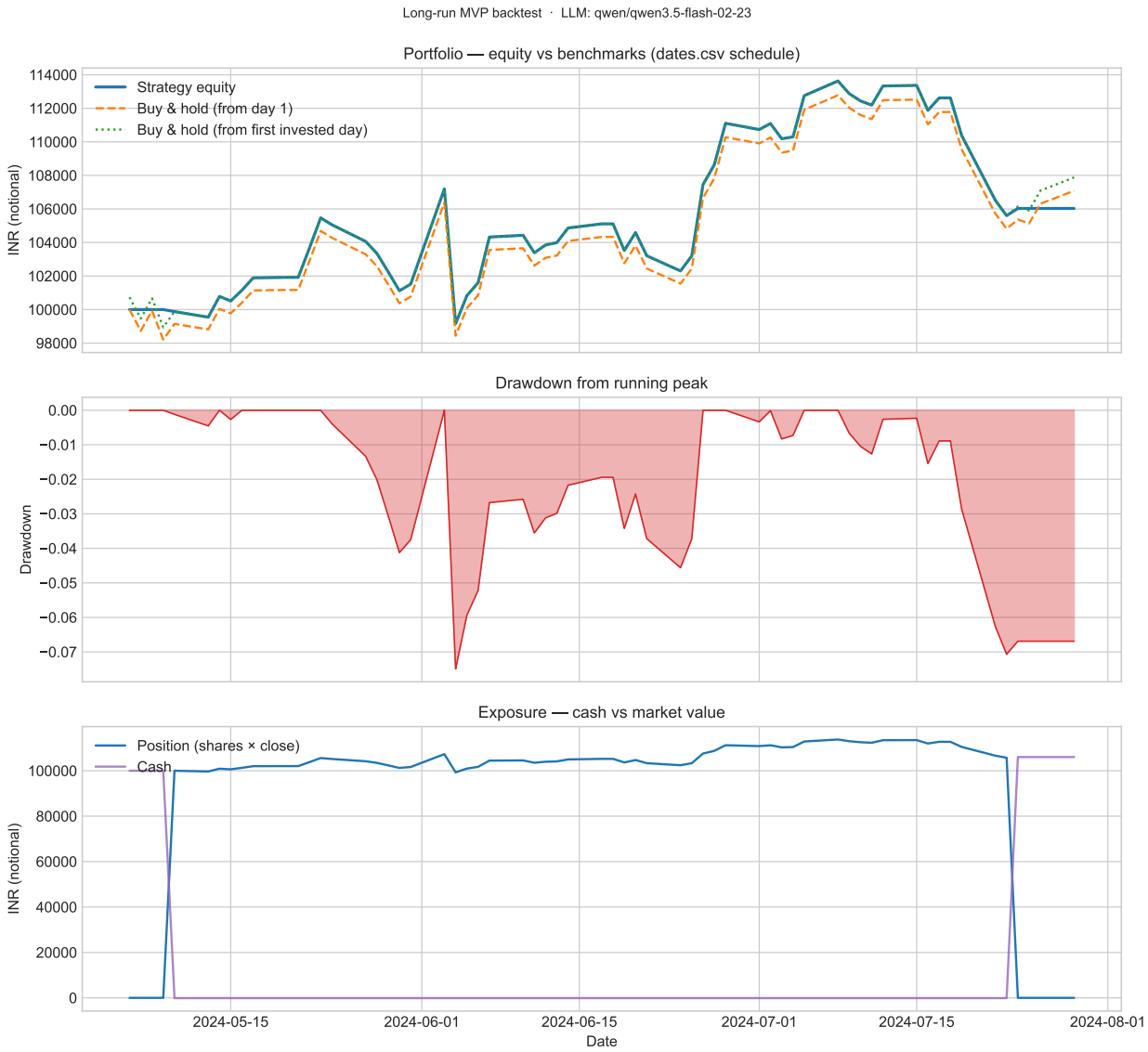

Figure 4: Figure 4. Longitudinal schedule run: portfolio equity vs benchmarks, drawdown, and cash vs position value.

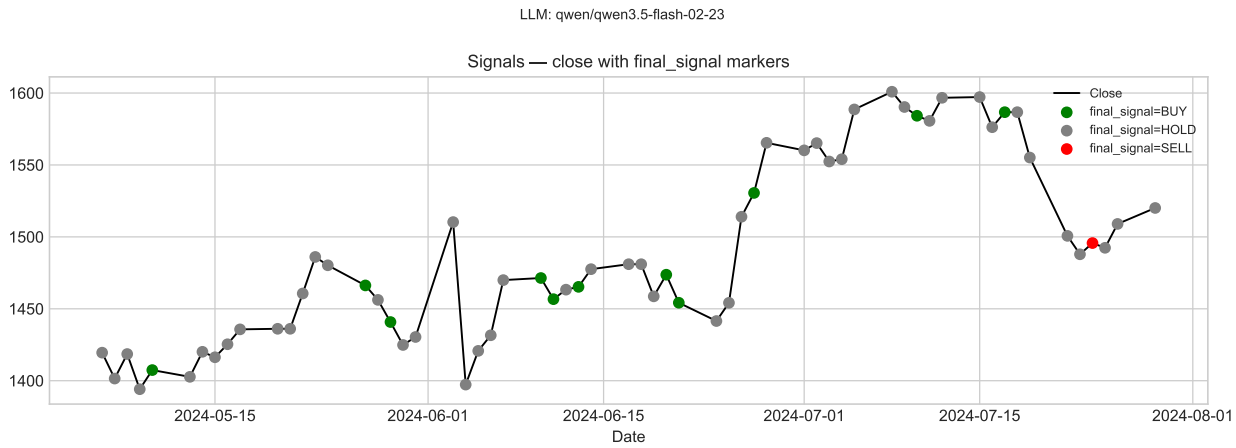

Figure 5: Figure 5. Longitudinal run: close price with final_signal markers.

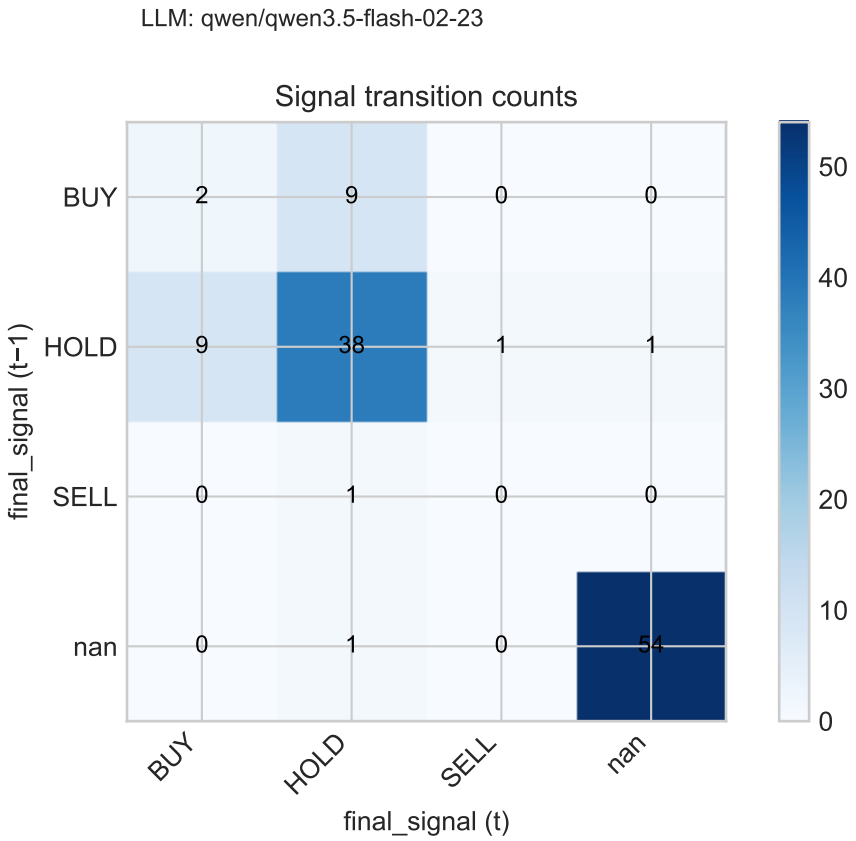

Figure 6: Figure 6. Longitudinal run: signal transition counts.

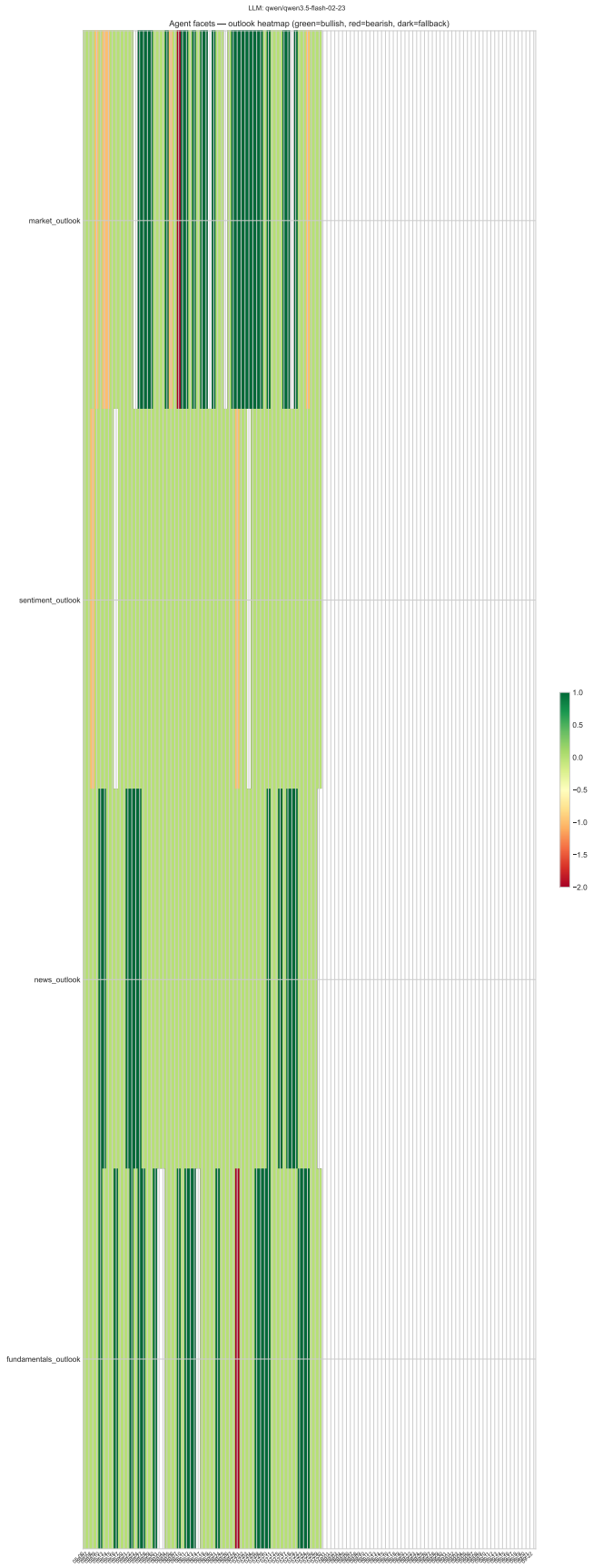

Figure 7: Figure 7. Longitudinal run: agent outlook heatmap.

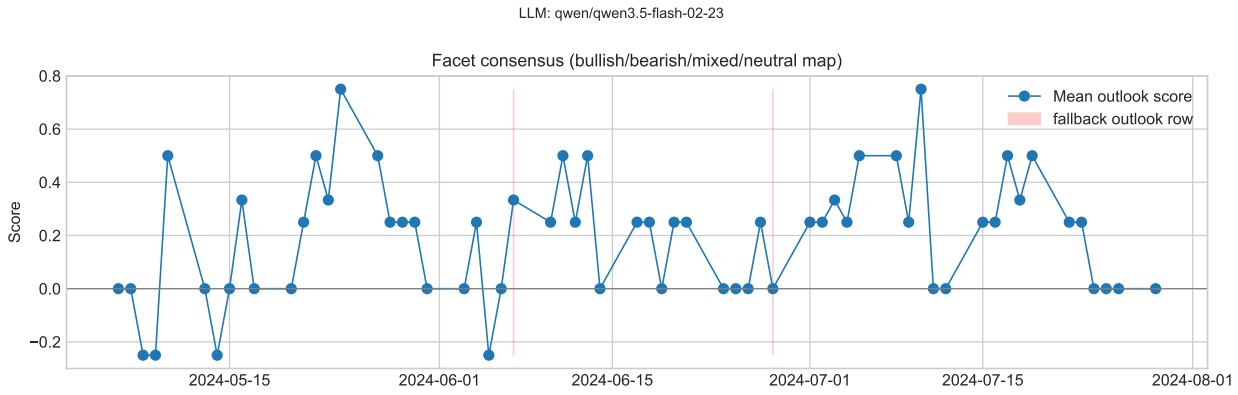

Figure 8: Figure 8. Longitudinal run: facet consensus score.

0.6 Contributions

Each item maps to memory-aligned contribution IDs **S1**–**S5** and claim-map IDs **M1**–**M6** in the repository documentation.

1. **Multi-agent decision graph.** We describe a LangGraph-orchestrated pipeline that routes single-stock decisions through analyst modules, investment and risk debate stages, and downstream trading logic so that depth and participation can be varied within one codepath (S1 / M1).

2. **Controlled ablations on the same engine.** We contribute preset ablation modes that toggle which analysts and debate stages run, selected via `PAPER_ABLATION`, supporting reproducible comparisons of pipeline depth without maintaining parallel implementations (S2 / M2).

3. **Structured stage outputs.** We integrate schema-constrained LLM outputs (Pydantic models and structured parsing fallbacks) so that intermediate decisions and rationales are machine-checkable and easy to retain in graph state and logs (S3 / M3).

4. **Point-in-time data policy and optional anonymization.** We document a simulation-date cap and vendor routing that bound requests to information available on each trade date, and optional ticker anonymization for experiments that seek to

limit direct ticker exposure in model-visible text (S4 / M4–M5).

5. **Backtest harness and analysis artifacts.** We pair the agent stack with a paper-ledger backtest, configurable fees, standard return and risk metrics, and CSV-oriented artifacts produced by the shipped MVP and ablation driver scripts, enabling evaluation from stored runs and notebooks (S5 / M6).

Not claimed here: superiority over proprietary or undisclosed systems; new predictability theory; or metrics that require features listed only as future work in the repository roadmap.

Acknowledgements

Not applicable.

References

Xiao, Y., Sun, E., Luo, D. and Wang, W., 2024. TradingAgents: Multi-Agents LLM Financial Trading Framework. *arXiv preprint* arXiv:2412.20138 [q-fin.TR]. Available at: https://arxiv.org/abs/2412.20138 (Accessed: 30 April 2026).

Yang, H., Liu, X.-Y. and Wang, C.D., 2023. FinGPT: Open-Source Financial Large Language Models. *arXiv preprint* arXiv:2306.06031 [q-fin.ST]. Available at: https://arxiv.org/abs/2306.06031 (Accessed: 30 April 2026).

Yu, Y., Li, H., Chen, Z., Jiang, Y., Li, Y., Zhang, D., Liu, R., Suchow, J.W. and Khashanah, K., 2023. FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design. *arXiv preprint* arXiv:2311.13743 [q-fin.CP]. Available at: https://arxiv.org/abs/2311.13743 (Accessed: 30 April 2026).